

Drake AI Enterprise Platform

Exact technical stack, module explanations, section-level methods, and model usage from the repository code.

1. Project-Level Stack

Project type	Full-stack petroleum intelligence platform for LAS processing, petrophysics, AI prediction, uncertainty, seismic enhancement, CCUS, geothermal, production analytics, reporting, and AI copilot chat.
Frontend	React 18, TypeScript, Vite 5, Tailwind CSS, React Router DOM, TanStack Query, Zustand, Axios, Plotly.js, Recharts, React Dropzone, React Hot Toast, lucide-react.
Backend	FastAPI, Python, Pydantic, SQLAlchemy, Alembic, uvicorn, python-jose JWT, passlib bcrypt, python-multipart, aiofiles.
Data and storage	SQLite local default, PostgreSQL 15 in Docker, Redis cache/session store, MinIO/S3-compatible configuration, local uploads directory.
AI/ML libraries	scikit-learn, optional XGBoost, TensorFlow/Keras, PyWavelets, NumPy, pandas, SciPy, lasio, welly, segyio, Plotly, matplotlib.
Container stack	Docker and Docker Compose services for frontend, backend, postgres, redis, and minio.

Important Interpretation

Not every section uses a machine-learning model. Crossplot and Histogram are statistical/visual sections. Missing-log prediction, facies, supervised tops, seismic enhancement, production lift failure, GPT, and AI parameter prediction contain explicit model or algorithm usage.

2. Complete UI Technical Stack

UI section	Files / routes	Technology used	What it does
App shell	frontend/src/App.tsx, components/layout/*	React 18, TypeScript, React Router DOM, MainLayout, private route token check	Defines the authenticated workspace and all module routes.
State / API	frontend/src/store/index.ts, frontend/src/services/api.ts	Zustand, Axios, TanStack Query, local/session storage helpers	Stores auth/project/module state and sends requests to FastAPI endpoints.
Styling	frontend/src/index.css, tailwind.config.js, postcss.config.js	Tailwind CSS, PostCSS, Autoprefixer, custom CSS, inline component styles	Controls application look, layout, page panels, controls, and dashboards.
Visualization UI	UIOnlyModulePage.tsx, components/petrophysics/*	Plotly.js, Recharts, custom tables, upload panels, Plotly export controls	Shows logs, crossplots, histograms, seismic heatmaps, prediction curves, and analytics tables.
Uploads	React Dropzone usage, API service wrappers	react-dropzone, Axios multipart uploads	Uploads LAS, SEG-Y, NPY, CSV, TXT, XLSX files depending on module.
Notifications/icons	Global Toaster, page buttons	react-hot-toast, lucide-react	Shows success/error messages and icon-based controls.

UI Explanation

The UI is a Vite React TypeScript app. React Router defines module pages. Zustand stores auth/project/module state. Axios service wrappers call the FastAPI backend. Plotly and Recharts render technical charts. Tailwind and custom styles define layout and visual design.

3. Backend API Stack

API group	Main code	Purpose	Model / method
/api/auth	backend/app/api/auth.py	Login and token generation.	No ML. JWT + bcrypt authentication.
/api/projects	backend/app/api/projects.py	Project CRUD and active project workflows.	No ML. SQLAlchemy data operations.
/api/wells	backend/app/api/wells.py	Well metadata and well listing.	No ML. SQLAlchemy data operations.
/api/curves	backend/app/api/curves.py	Curve listing and curve data retrieval.	No ML. Returns stored measured/predicted curve arrays.
/api/files	backend/app/api/files.py	Upload and parse LAS/well files.	No ML. File parsing and curve persistence.
/api/ai	backend/app/api/ai_jobs.py, backend/app/ml/petrophysics.py	Creates and tracks AI job records.	Returns demo/result model labels including Random Forest, Gradient Boosting, CNN boundary detector, and LSTM + RF ensemble.
/api/gpt	backend/app/api/gpt.py	Drake GPT copilot with well context.	Anthropic claude-sonnet-4-20250514; fallback: Drake GPT Local rule-based responses.
/api/petrophysics	backend/app/api/petrophysics.py	LAS sessions, crossplot, histogram, missing logs, predictions, uncertainty, facies, formation tops.	Uses statistics, scikit-learn regressors/classifiers, KMeans, SciPy signal methods, and prediction engines.
/api/seismic	backend/app/api/seismic.py, backend/app/ml/seismic_enhancer.py	Seismic file upload, inspect, frequency enhancement.	Conv1D U-Net, PyWavelets + RandomForest, Conv1D autoencoder, FFT fallback.
/api/geothermal	backend/app/api/geothermal.py, backend/app/ml/geothermal.py	Geothermal LAS screening and exports.	Rule/calculation based geothermal metrics; no named trained ML model.
/api/production	backend/app/api/production.py, backend/app/ml/production_intelligence.py	Production analytics, optimizer, failure risk, decline, workover ranking.	RandomForestClassifier for lift failure when data supports it; otherwise rule-based fallback.
/api/ccus	backend/ccus/router.py, backend/ccus/core.py	Preliminary CCS screening and XLSX export.	Rule-based petrophysical cutoff and quality scoring model.

4. Petrophysics Sections

This page separates the petrophysics sections clearly, including Crossplot and Histogram.

Section	Technical stack	Model / method used	Clear code explanation
LAS upload/session	FastAPI UploadFile, lasio, pandas, NumPy, RedisBackedSessionStore, pickle, uuid	No ML model.	Reads LAS bytes, repairs/normalizes LAS data, converts curves to numeric pandas DataFrame, stores session by id, returns curve names, depth range, rows, and metadata.
Crossplot	backend/app/api/petrophysics.py generate_crossplot; pandas, NumPy, Plotly scattergl JSON	No ML model. Uses filtering and Pearson correlation via np.corrcoef.	Takes x/y curves, optional color curve, depth window, and linear/log scaling. Drops invalid rows, computes correlation/statistics, builds Plotly scattergl points colored by depth or selected curve.
Histogram	backend/app/api/petrophysics.py generate_histogram; NumPy histogram, SciPy gaussian_kde, SciPy stats, Plotly-ready arrays	No ML model. Statistical distribution analysis only.	Parses selected curve values, applies depth/range/log filters, calculates bins, KDE curve, mean/median/std/percentiles, skewness, kurtosis, z-score outlier percentage, completeness, quality score, and distribution label.
Missing log prediction	ExtraTreesRegressor, RandomForestRegressor, GradientBoostingRegressor, pandas, NumPy, r2_score	Selectable regressors: Extra Trees, Random Forest, Gradient Boosting.	Builds features from available LAS curves and depth, trains on valid samples where target curve exists, predicts missing intervals, returns predicted count, model name, R2-style metrics, figure, CSV, and filled LAS output.
AI parameter prediction	backend/app/ml/drake_prediction_standalone.py; NumPy, pandas, RandomForestRegressor, XGBRegressor, GradientBoostingRegressor, DecisionTreeRegressor	Empirical formulas; Random Forest AI; XGBoost AI; Gradient Boosting AI; Decision Tree AI.	Computes VSH, porosity, water saturation, permeability, lithology, confidence, and P10/P50/P90 rows. ML mode trains pseudo-supervised/ensemble models from available LAS-derived inputs; empirical mode uses petrophysical formulas.
AI uncertainty	backend/app/ml/drake_uncertainty*.py, prediction sections, NumPy/pandas, optional model spread	Random Forest tree/ensemble spread; XGBoost/boosting bootstrap spread; percent-based fallback.	Builds uncertainty records around porosity and saturation predictions, returns P10/P50/P90 curves, uncertainty spread, confidence/reliability, and summary cards.
Facies classification	backend/app/ml/mini_petrophysics_toolbox.py; StandardScaler, KMeans, MiniBatchKMeans, RandomForestClassifier, Plotly	Unsupervised: KMeans/MiniBatchKMeans. Supervised: RandomForestClassifier with 150 estimators.	Requires depth and at least three predictor curves. In unsupervised mode clusters numeric log features into facies groups. In supervised mode trains Random Forest from a selected facies label column and reports accuracy/classification report.
Formation tops	SciPy savgol_filter, scipy.signal.find_peaks, NumPy gradients, RandomForestClassifier for supervised mode, Plotly	Unsupervised gradient/peak detector. Supervised mode uses RandomForestClassifier with 300 estimators.	Detects stratigraphic breaks by smoothing curves, computing normalized multi-curve gradients, finding peaks, assigning confidence and evidence curve. If reference tops are provided, supervised mode learns formation labels near tops.
Auto splicer	backend/app/ml/petrophysics.py job workflow	No external trained model. Returned label: Depth-Match + Overlap Merge.	Demo/job workflow for aligning and merging intervals based on depth overlap.

5. Seismic Module

Section	Technical stack	Model / method used	Clear code explanation
Upload/inspect	FastAPI UploadFile, pathlib, shutil, segyio, NumPy, CSV/TXT/NPY loading	No ML model.	Accepts .sgy, .segy, .npy, .csv, .txt. Stores file under uploads/seismic and inspects trace count, sample count, interval, and data shape.
2D enhancement	TensorFlow/Keras Conv1D, Conv1DTranspose, MaxPooling1D, Lambda shape matching, MinMaxScaler	2D TensorFlow Conv1D U-Net.	Scales traces, trains denoising/enhancement U-Net-like Conv1D model, predicts enhanced trace section, applies gain, and returns enhanced panel/metrics.
3D low frequency	PyWavelets wavedec/waverec, SciPy uniform_filter1d, RandomForestRegressor	3D PyWavelets + RandomForest low-frequency ML.	Decomposes traces into wavelet coefficients, smooths low-frequency coefficients, trains RandomForestRegressor to enhance low-frequency content, reconstructs volume.
3D high frequency	TensorFlow/Keras Sequential Conv1D autoencoder, Dense layers, MinMaxScaler, FFT	3D TensorFlow Conv1D frequency-band autoencoder.	Extracts FFT frequency-band magnitudes, trains Conv1D autoencoder to denoise/enhance band magnitudes, reinserts magnitude/phase into spectrum, inverse FFTs to time domain.
Combined workflow	PyWavelets + RandomForest + Conv1D autoencoder + FFT combine	3D combined RandomForest low-frequency + Conv1D DL high-frequency.	Runs both low-frequency and high-frequency enhancement and combines them using a frequency cutoff.
Fallback	NumPy FFT	FFT band enhancement fallback.	If TensorFlow/PyWavelets/scikit-learn runtime path fails, boosts selected frequency band with FFT so the module still returns a valid enhanced result shape.

Seismic Result Explanation

The seismic result payload returns model_stack, metrics, preview rows, original/enhanced/difference sections, frequency spectrum, and optional enhanced SEG-Y output. If advanced dependencies fail, the FFT fallback still returns a valid result.

6. Other Domain Modules

Module	Technical stack	Model / method used	Clear code explanation
Drake GPT	FastAPI, Anthropic SDK, SQLAlchemy well context, Pydantic request/response	Primary LLM: claude-sonnet-4-20250514. Fallback: Drake GPT Local rule-based logic.	Builds a system prompt from well metadata, curves, formation tops, and completed jobs. Sends user chat to Anthropic when key is configured; otherwise returns local rule-based petrophysics answers.
Production intelligence	pandas, NumPy, scikit-learn RandomForestClassifier, train_test_split, accuracy_score	Artificial lift failure uses RandomForestClassifier with 120 estimators if enough labelled failure data exists; otherwise rule-based fallback.	Cleans production data, calculates water cut/GOR/drawdown, predicts lift failure risk, ranks optimization actions, fits exponential decline, computes health score, downtime loss, and workover priority.
Geothermal screening	Custom LAS parser, pandas, NumPy, matplotlib PNG export	No named trained ML model. Rule/calculation based.	Maps LAS curves, estimates temperature if absent, computes geothermal gradient, VSH, porosity, permeability, reservoir quality score, hot-zone score, heat flow, thermal index, target intervals, map payload, CSV/JSON/PNG exports.
CCUS screening	Custom LAS parser, math/stat helpers, Plotly-compatible tracks, XLSX writer	No ML model. Deterministic cutoff/rule-based scoring.	Uses GR, RHOB, NPHI, RT, PHIE, VSH, permeability proxy and user cutoffs to identify CO2 storage candidates, poor intervals, seal/caprock candidates, reservoir-seal pairs, and quality scores.
Reports/export	FastAPI FileResponse, fpdf2, openpyxl, pandas, report utilities	No ML model.	Packages generated results into PDF/spreadsheet/CSV/JSON/LAS-style outputs depending on module.
Standalone Drake_Uncertainty	Flask, Jinja templates, Plotly, lasio, pandas, NumPy, SciPy, scikit-learn, optional XGBoost	RandomForestRegressor, XGBRegressor, GradientBoostingRegressor, DecisionTreeRegressor, RandomForestClassifier helpers, empirical formulas.	Separate dashboard for LAS upload, prediction, uncertainty, visualization, export, and analysis history. It mirrors many backend prediction ideas in a standalone Flask app.

7. Exact Model / Algorithm Summary

Model / algorithm	Where used	Purpose
claude-sonnet-4-20250514	backend/app/api/gpt.py	LLM copilot answer generation with well context.
Drake GPT Local	backend/app/api/gpt.py	Rule-based fallback when Anthropic key is missing or call fails.
ExtraTreesRegressor	Missing log prediction	Predict missing LAS target curve samples.
RandomForestRegressor	Missing logs, AI parameter prediction, seismic low-frequency enhancement	Curve regression, ensemble uncertainty spread, or wavelet coefficient enhancement.
GradientBoostingRegressor	Missing logs, AI parameter prediction	Alternative regression model for target curve/petrophysical output prediction.
XGBRegressor	AI parameter prediction / standalone uncertainty app	Optional boosted regression model for predictions and bootstrap uncertainty.
DecisionTreeRegressor	AI parameter prediction / standalone app	Alternative simpler tree model for prediction and uncertainty bands.
KMeans / MiniBatchKMeans	Facies classification	Unsupervised facies cluster assignment from selected logs.
RandomForestClassifier	Facies, formation tops supervised mode, production lift failure	Supervised facies/formation labels and production failure risk.
SciPy savgol_filter + find_peaks	Formation tops	Detect curve-gradient peaks as formation top candidates.
SciPy gaussian_kde	Histogram	Smooth KDE curve over histogram values; not ML.
TensorFlow Conv1D U-Net	Seismic 2D enhancement	Deep-learning trace enhancement.
TensorFlow Conv1D autoencoder	Seismic high-frequency enhancement	Frequency-band denoising/enhancement.
PyWavelets + RandomForest	Seismic low-frequency enhancement	Wavelet coefficient enhancement for 3D seismic.
FFT band enhancement	Seismic fallback	Dependency-light frequency band gain when advanced models are unavailable.
Rule-based petrophysical formulas	CCUS, geothermal, empirical prediction, production fallback	Deterministic calculations from logs and thresholds.

8. Main Code References

Area	Main files
Frontend routes/UI	frontend/src/App.tsx; frontend/src/pages/UIOnlyModulePage.tsx; frontend/src/components/petrophysics/*; frontend/src/services/api.ts
Backend entry/config	backend/app/main.py; backend/app/core/config.py; backend/app/core/security.py; backend/app/core/database.py
Petrophysics API	backend/app/api/petrophysics.py
Prediction/uncertainty engines	backend/app/ml/drake_prediction_standalone.py; backend/app/ml/drake_uncertainty.py; backend/app/ml/drake_uncertainty_standalone.py
Facies/tops toolbox	backend/app/ml/mini_petrophysics_toolbox.py
Seismic	backend/app/api/seismic.py; backend/app/ml/seismic_enhancer.py
Production	backend/app/api/production.py; backend/app/ml/production_intelligence.py
Geothermal	backend/app/api/geothermal.py; backend/app/ml/geothermal.py
CCUS	backend/ccus/router.py; backend/ccus/core.py
GPT	backend/app/api/gpt.py
Standalone app	Drake_Uncertainty/app.py; Drake_Uncertainty/utlis/ml_utils.py; Drake_Uncertainty/templates/*; Drake_Uncertainty/static/*

Final Notes

This PDF is based on static code inspection. Where the code returns demo-style labels, the report states them as returned labels. Where no ML model is used, the report explicitly says no ML model and names the statistical, formula-based, or rule-based method instead.